

ESEIT - PARCIAL No 3 – Parcial Práctico INGENIERÍA DEL SOFTWARE SEM2-2025
ING. USB. RITTNER CASTRO

NOMBRES Y APELLIDOS: _____

IDENTIFICACIÓN: _____ CODIGO ESEIT: _____

REGISTRO MATERIA CODIGO: _____ FECHA: _____

Parcial Avanzado – Ingeniería de Software (Java)

Instrucciones:

- Cada pregunta tiene una única respuesta correcta.
- Justifique su elección con base en conceptos teóricos y prácticos del desarrollo en Java.

1. ¿Cuál de los siguientes principios SOLID establece que “una clase debe tener una única razón para cambiar”?

- A) Principio de Inversión de Dependencias
- B) Principio de Sustitución de Liskov
- C) Principio de Responsabilidad Única
- D) Principio de Segregación de Interfaces

2. ¿Qué patrón de diseño se utiliza comúnmente en Spring Framework para gestionar la creación de objetos?

- A) Observador
- B) Fábrica (Factory)
- C) Singleton
- D) Inyección de Dependencias

3. ¿Qué característica de Java permite la ejecución de múltiples hilos simultáneamente?

- A) Multitarea
- B) Concurrencia
- C) Multiprocesamiento
- D) Paralelismo

4. ¿Qué anotación de JUnit 5 se usa para marcar un método como prueba?

- A) @Test
- B) @TestCase

- C) @Unit
 - D) @Run
-

5. ¿Cuál es el propósito del archivo pom.xml en un proyecto Maven?

- A) Almacenar código fuente
 - B) Configurar el servidor de aplicaciones
 - C) Definir dependencias, plugins y metadatos del proyecto
 - D) Ejecutar pruebas unitarias
-

6. ¿Qué patrón de diseño permite agregar comportamiento adicional a un objeto de forma dinámica sin alterar su estructura?

- A) Decorador
 - B) Adaptador
 - C) Estrategia
 - D) Comando
-

7. ¿Qué interfaz de Java se utiliza comúnmente como contrato para implementar comparaciones personalizadas?

- A) Comparable
 - B) Comparator
 - C) Ambas A y B
 - D) Iterable
-

8. ¿Cuál es el propósito del patrón “Repository” en arquitecturas limpias?

- A) Gestionar la lógica de negocio
 - B) Abstraer el acceso a datos
 - C) Controlar la autenticación del usuario
 - D) Manejar excepciones
-

9. ¿Qué característica de Java 8 permite implementar métodos dentro de interfaces?

- A) Métodos abstractos
 - B) Métodos estáticos
 - C) Métodos por defecto (default)
 - D) Métodos privados
-

10. ¿Qué framework de Spring se usa para construir aplicaciones web reactivas no bloqueantes?

- A) Spring MVC
 - B) Spring Boot
 - C) Spring WebFlux
 - D) Spring Security
-

11. ¿Qué hace la anotación @SpringBootApplication?

- A) Solo habilita pruebas
 - B) Es un alias de @ComponentScan
 - C) Combina @Configuration, @EnableAutoConfiguration y @ComponentScan
 - D) Solo configura el servidor web
-

12. ¿Cuál de las siguientes NO es una característica del desarrollo basado en pruebas (TDD)?

- A) Escribir pruebas antes del código de producción
 - B) Refactorizar continuamente
 - C) Asegurar 100% de cobertura de código
 - D) Ciclo “Red-Green-Refactor”
-

13. ¿Qué clase de Java se utiliza para trabajar con fechas y horas de forma inmutable desde Java 8?

- A) Date
 - B) Calendar
 - C) LocalDateTime
 - D) Timestamp
-

14. ¿Qué principio SOLID se viola si una subclase lanza una excepción que la clase base no lanza?

- A) Principio de Abierto/Cerrado
 - B) Principio de Sustitución de Liskov
 - C) Principio de Segregación de Interfaces
 - D) Principio de Inversión de Dependencias
-

15. ¿Cuál de los siguientes NO es un framework o biblioteca de pruebas en Java?

- A) Mockito
- B) JUnit

- C) TestNG
 - D) Hibernate
-

16. ¿Qué patrón se utiliza para garantizar que una clase tenga una única instancia y acceso global a ella?

- A) Fábrica
 - B) Builder
 - C) Singleton
 - D) Prototipo
-

17. ¿Qué característica de Java permite que una clase implemente múltiples interfaces?

- A) Herencia múltiple
 - B) Polimorfismo
 - C) Implementación múltiple
 - D) Encapsulamiento
-

18. ¿Cuál de las siguientes anotaciones de Spring se usa para inyectar una dependencia?

- A) @Autowired
 - B) @Inject
 - C) Ambas A y B
 - D) @Component
-

19. ¿Qué es un “bean” en Spring?

- A) Un archivo de configuración
 - B) Un objeto gestionado por el contenedor de Spring
 - C) Un tipo de excepción
 - D) Una clase de prueba
-

20. ¿Qué interfaz funcional de Java se usa para representar una operación que acepta un argumento y no devuelve nada?

- A) Function<T, R>
 - B) Consumer<T>
 - C) Supplier<T>
 - D) Predicate<T>
-

21. ¿Qué patrón arquitectónico separa la lógica de presentación de la lógica de negocio?

- A) Microservicios
 - B) MVC (Modelo-Vista-Controlador)
 - C) Event-Driven
 - D) Capas (Layers)
-

22. ¿Qué herramienta de construcción se considera más moderna que Maven y permite scripts en Groovy o Kotlin?

- A) Ant
 - B) Ivy
 - C) Gradle
 - D) SBT
-

23. ¿Cuál de las siguientes NO es una característica de las aplicaciones “12-factor”?

- A) Configuración mediante variables de entorno
 - B) Dependencia de archivos locales para estado
 - C) Logs como streams de eventos
 - D) Construcción, liberación y ejecución como fases separadas
-

24. ¿Qué clase de Java se recomienda usar para operaciones matemáticas con alta precisión decimal?

- A) float
 - B) double
 - C) BigDecimal
 - D) Integer
-

25. ¿Qué permite la anotación @Transactional en Spring?

- A) Validar datos de entrada
 - B) Gestionar transacciones de base de datos automáticamente
 - C) Seguridad de métodos
 - D) Cacheo de resultados
-

26. ¿Cuál es el propósito del patrón “Builder”?

- A) Crear objetos complejos paso a paso
- B) Clonar objetos existentes

- C) Simplificar la serialización
 - D) Gestionar conexiones a BD
-

27. ¿Qué mecanismo de Java evita que dos hilos accedan simultáneamente a un bloque crítico?

- A) synchronized
 - B) volatile
 - C) transient
 - D) final
-

28. ¿Qué es un “mock” en pruebas unitarias?

- A) Una copia de seguridad de la base de datos
 - B) Un objeto simulado que imita el comportamiento de dependencias reales
 - C) Un tipo de error de compilación
 - D) Un patrón de diseño
-

29. ¿Qué anotación de JPA se usa para mapear una clase a una tabla de base de datos?

- A) @Entity
 - B) @Table
 - C) @Column
 - D) @Repository
-

30. ¿Qué característica de Java permite extender funcionalidades de clases existentes mediante métodos estáticos?

- A) Herencia
- B) Métodos de extensión (como en C#)
- C) Métodos estáticos en clases utilitarias
- D) Interfaces con métodos por defecto